Lab 1 — Greedy Heuristics for Selective TSP with Node Costs

1. Description of the problem

We are given a set of `n = 200` nodes (instances TSPA, TSPB in `data/TSPA.csv`, `data/TSPB.csv`). Each row is `x;y;cost`:

- `(x, y)` — position in the plane,
- `cost[i] >= 0` — price paid if node `i` is visited.

Goal: **select exactly** `k = ceil(n/2) = 100` **nodes and find a Hamiltonian cycle** through them minimizing:

```
f(tour) = sum_{i} dist[tour[i], tour[(i+1) mod k]] + sum_{i} cost[tour[i]]
```

where `dist` is the Euclidean distance **rounded to the nearest integer**:

```
dist[i,j] = round(sqrt((x_i-x_j)^2 + (y_i-y_j)^2))
```

Important methodological constraint:

- The distance matrix is precomputed once after loading the instance.
- Optimization methods may only access `dist` + `costs`, never `(x, y)` — so the method

also works for pure matrix instances. Coordinates are used only for visualization.

This is a **Selective / Orienteering-like TSP**: subset selection and permutation are optimized jointly. Node costs create a non-trivial trade-off: a cheap but remote node can be worse than an expensive but well-located node. Hence "nearest" below is never pure Euclidean distance.

Formally, for candidate c and insertion between `a -> b` (with edge `a->b` removed):

```
delta_cycle(c, a, b) = dist[a,c] + dist[c,b] - dist[a,b] + cost[c]
```

For open-path construction (nearest-neighbor variants), closure is ignored until the end; the insertion cost is path-insertion delta (no `-dist[last,first]` term), plus `cost[c]`.

2. Implemented algorithms — pseudocode

Common notation: n nodes, select `k`, `D` distance matrix, `C` node costs, `used[]` visited flag.

2.1 Random solution

```
RANDOM(n, k, rng):
   selected <- sample k distinct indices from {0..n-1} without replacement
   shuffle(selected) uniformly
   return selected as cycle order
```

2.2 Nearest neighbor — append at end only (`nearest_neighbor_at_end`)

Greedy open path. Start from fixed `s`, always extend the tail.

```
NN-END(s, D, C, k):
   tour <- [s]; used[s] <- true
   repeat k-1 times:
       last <- tour.last
       best <- +inf; best_v <- -1
       for each c not used:
           value <- D[last, c] + C[c]        # edge extension + node price
           if value < best:
               best <- value; best_v <- c
       tour.append(best_v); used[best_v] <- true
   return tour   # evaluated as cycle: closing edge added in f(tour)
```

2.3 Nearest neighbor — insert anywhere (`nearest_neighbor_anywhere`)

Greedy open path, but each new node is inserted at the cheapest position: before first, after last, or inside.

```
NN-ANYWHERE(s, D, C, k):
```

```
   tour <- [s]; used[s] <- true
   repeat k-1 times:
       best_val <- +inf; best_c <- -1; best_pos <- -1
       for each c not used:
           # position 0 (prepend): D[c, tour[0]]
           # position m (append):  D[tour[m-1], c]
           # position p in 1..m-1: D[tour[p-1],c] + D[c,tour[p]] - D[tour[p-1],tour[p]]
           val_c, pos_c <- min over all positions (insertion delta, no closure)
           val_c <- val_c + C[c]
           if val_c < best_val:
               best_val <- val_c; best_c <- c; best_pos <- pos_c
       tour.insert(best_pos, best_c); used[best_c] <- true
   return tour
```

2.4 Greedy cycle (`greedy_cycle`*)*

Maintains a **closed cycle** at every step. Insertion delta accounts for both broken edge and closure.

```
GREEDY-CYCLE(s, D, C, k):
   tour <- [s]; used[s] <- true
   repeat k-1 times:
       if |tour| == 1:
           #  s -> c -> s
           c* <- argmin_c  2*D[s,c] + C[c]
           tour.append(c*)
       else:
           m <- |tour|
           # edges are (tour[i] -> tour[(i+1) mod m])
           best_val <- +inf; best_c <- -1; best_edge <- -1
           for each c not used:
               for each edge i:
                   val <- D[tour[i],c] + D[c,tour[(i+1) mod m]] - D[tour[i],tour[(i+1) mod m]] + C[c]
                   keep minimum over i -> val_c, edge_c
               if val_c < best_val:
                   best_val <- val_c; best_c <- c; best_edge <- edge_c
           tour.insert(best_edge+1, best_c)
       mark used
   return tour
```

3. Computational experiment

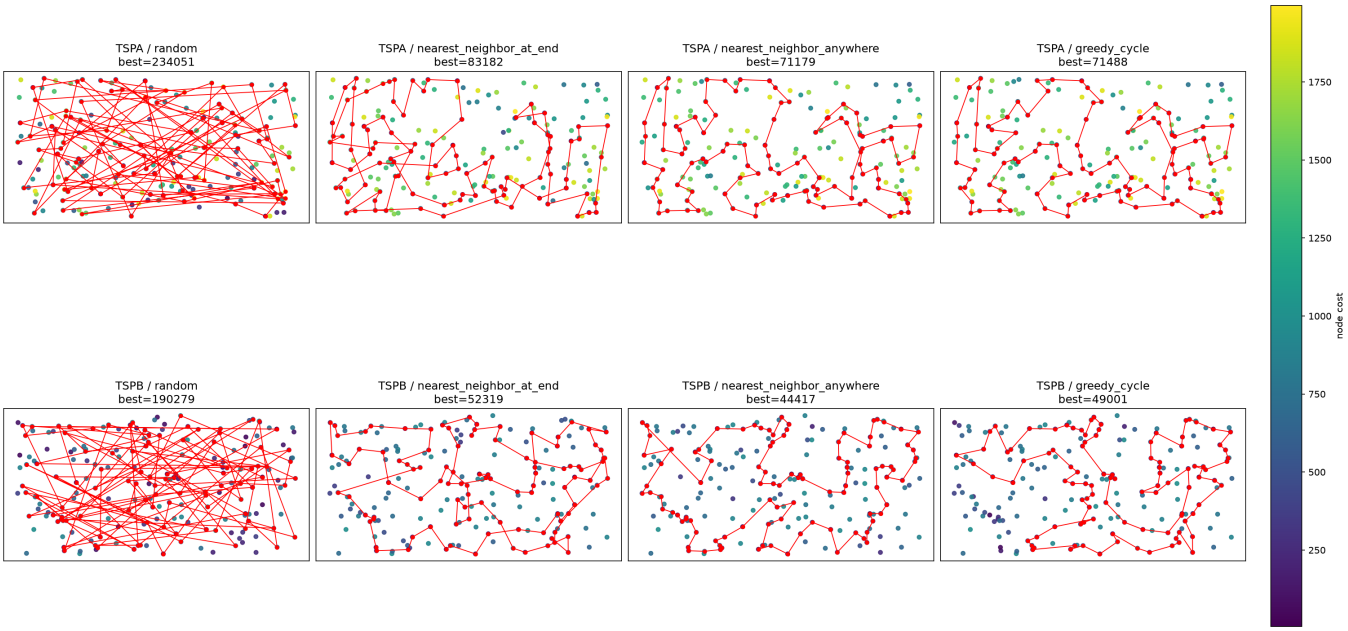
3.1 Results: min / max / mean

instance	method	min	max	mean
TSPA	random	234051	297098	262602.9
TSPA	nearest_neighbor_at_end	83182	89433	85108.5
TSPA	nearest_neighbor_anywhere	**71179**	75450	73178.4
TSPA	greedy_cycle	71488	**74410**	**72636.0**
TSPB	random	190279	238119	213699.7
TSPB	nearest_neighbor_at_end	52319	59030	54390.4
TSPB	nearest_neighbor_anywhere	**44417**	**53438**	**45870.3**
TSPB	greedy_cycle	49001	57324	51400.6

3.2 Visualization of best solutions

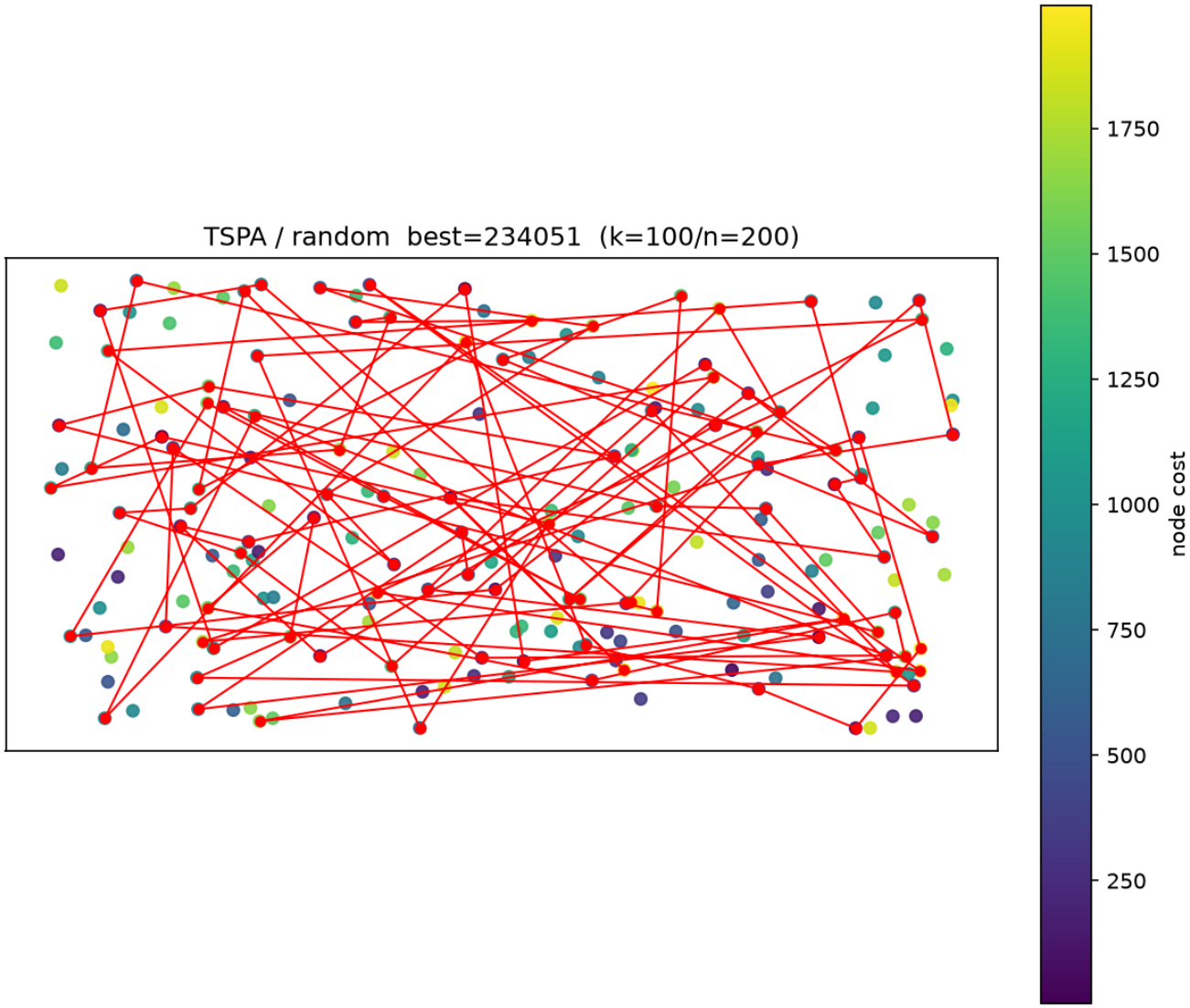
Common encoding in all plots: background dots = all 200 nodes, color (`viridis`) = node cost (see colorbar); red polyline + dots = best Hamiltonian cycle for that instance/method (closed, `k=100` nodes).
Overview grid (2 instances × 4 methods):

Best tours grid

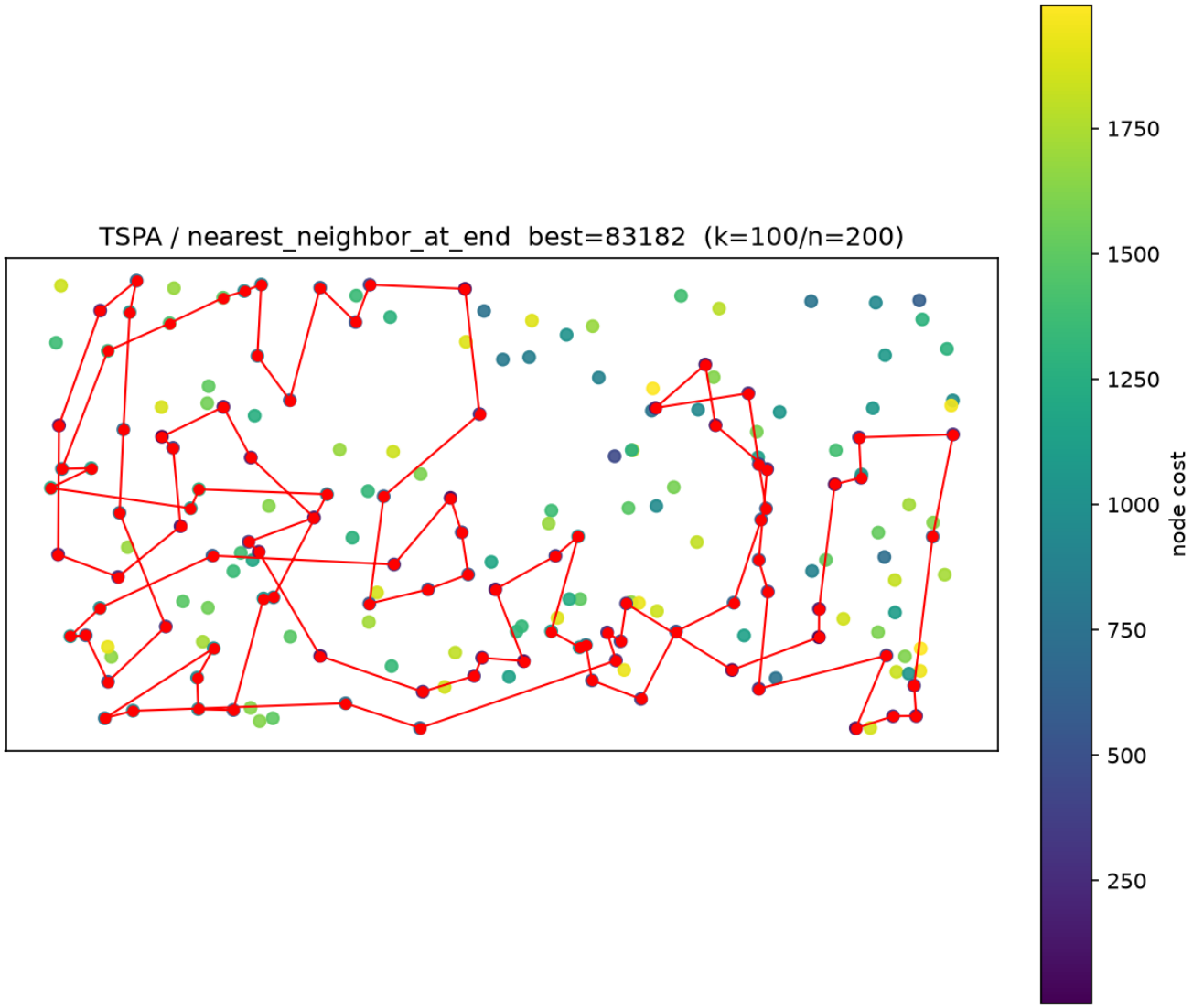


Individual best tours:

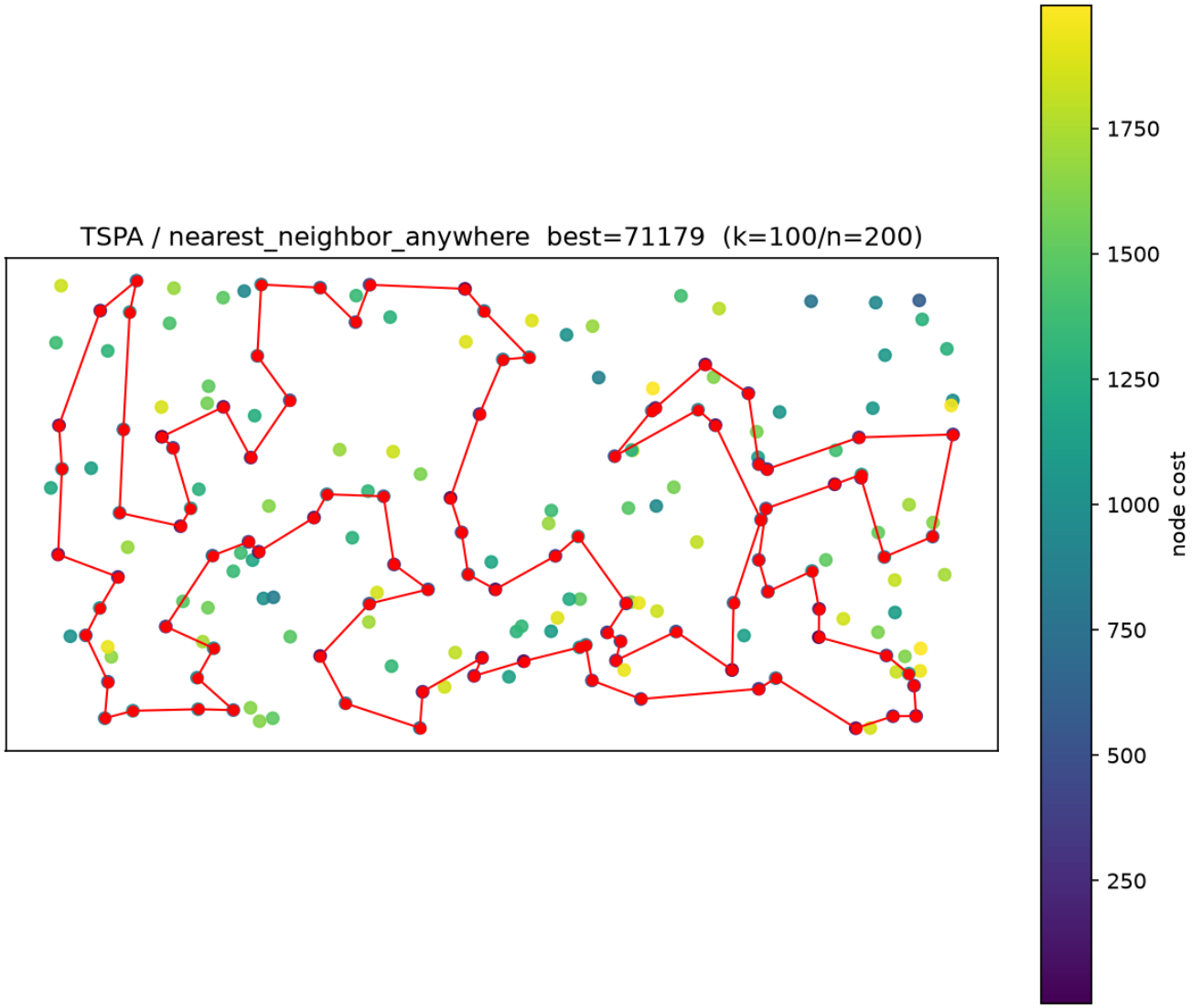
TSPA / random — best=234051



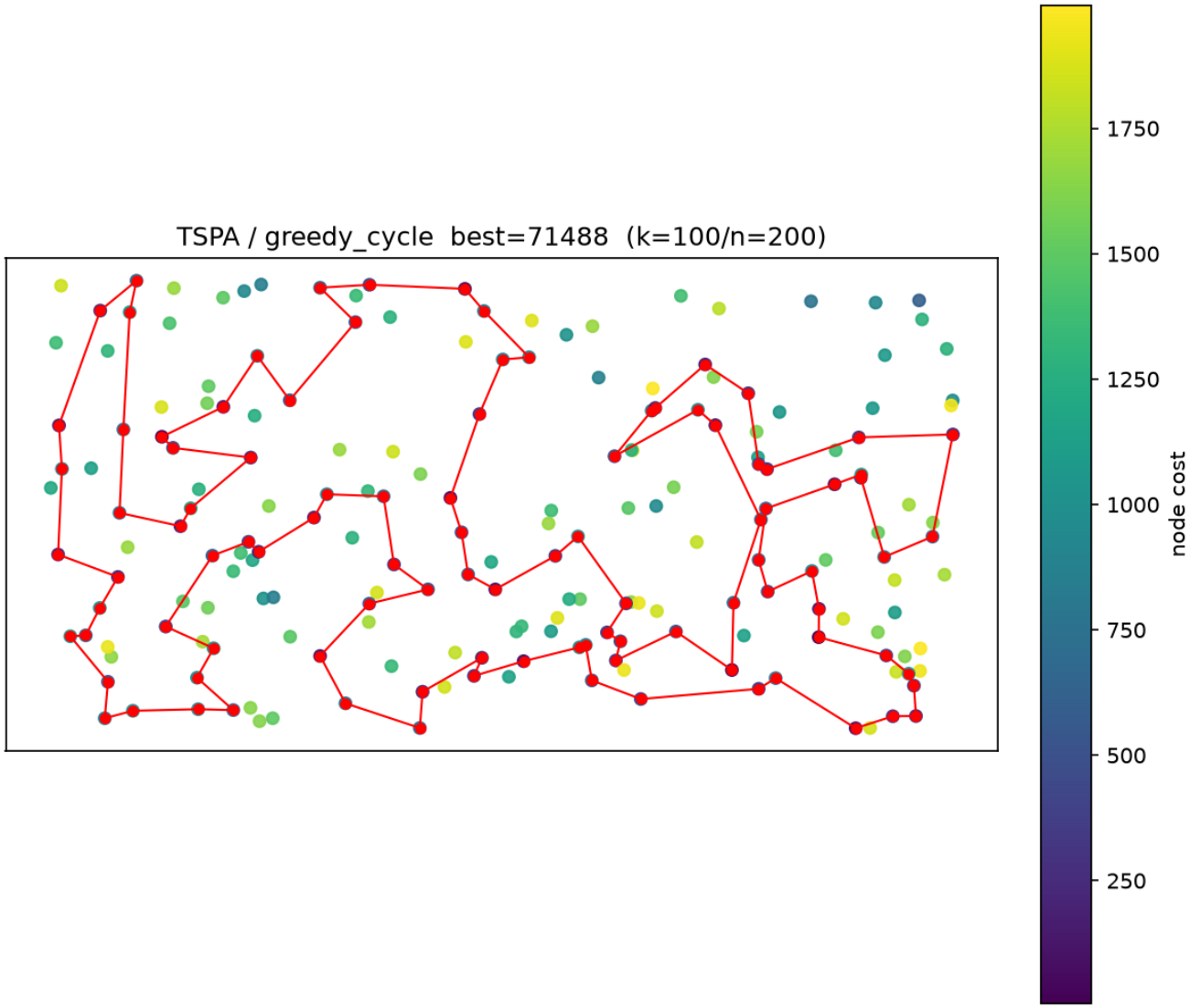
TSPA / nearest_neighbor_at_end — best=83182



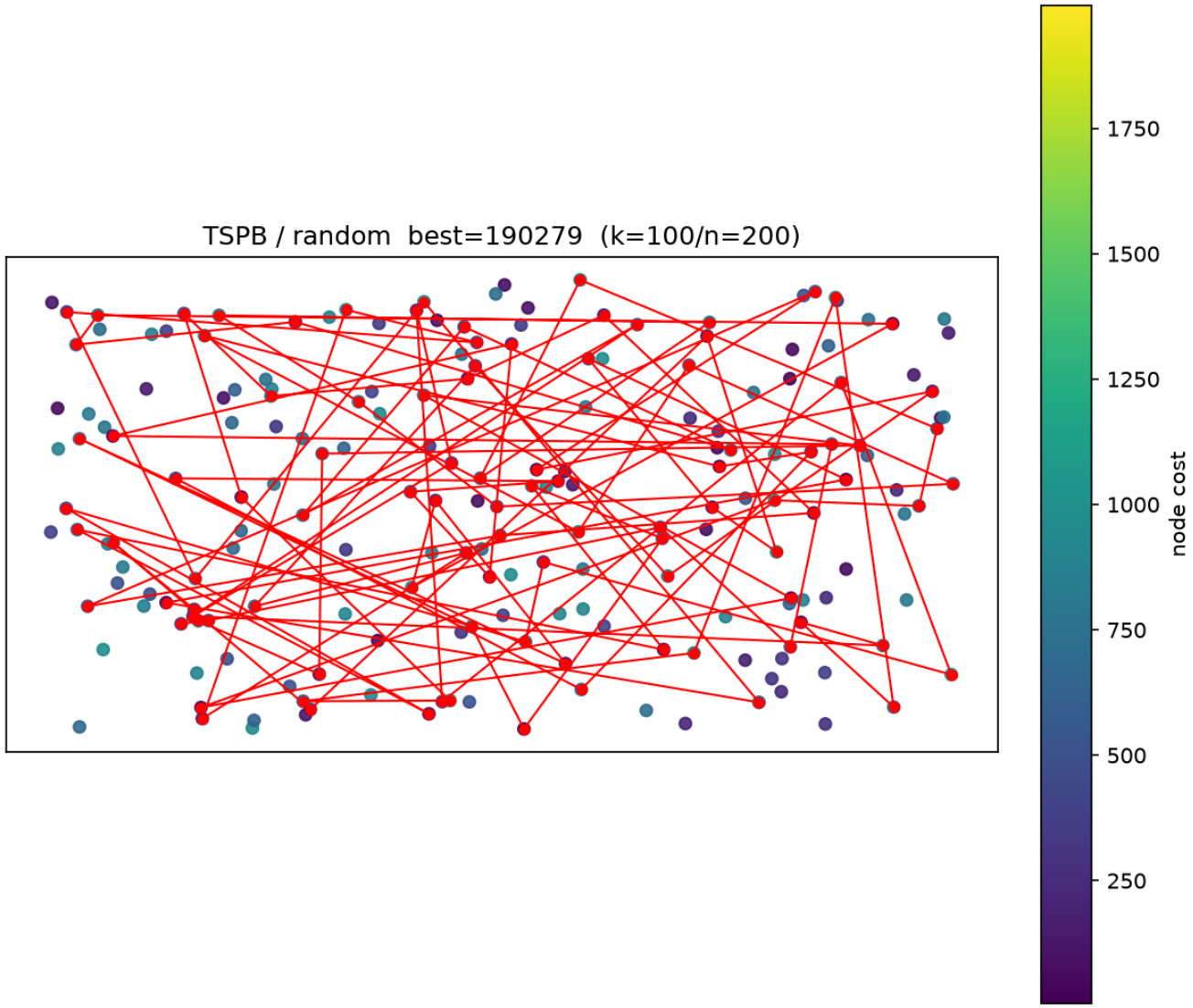
TSPA / nearest_neighbor_anywhere — best=71179



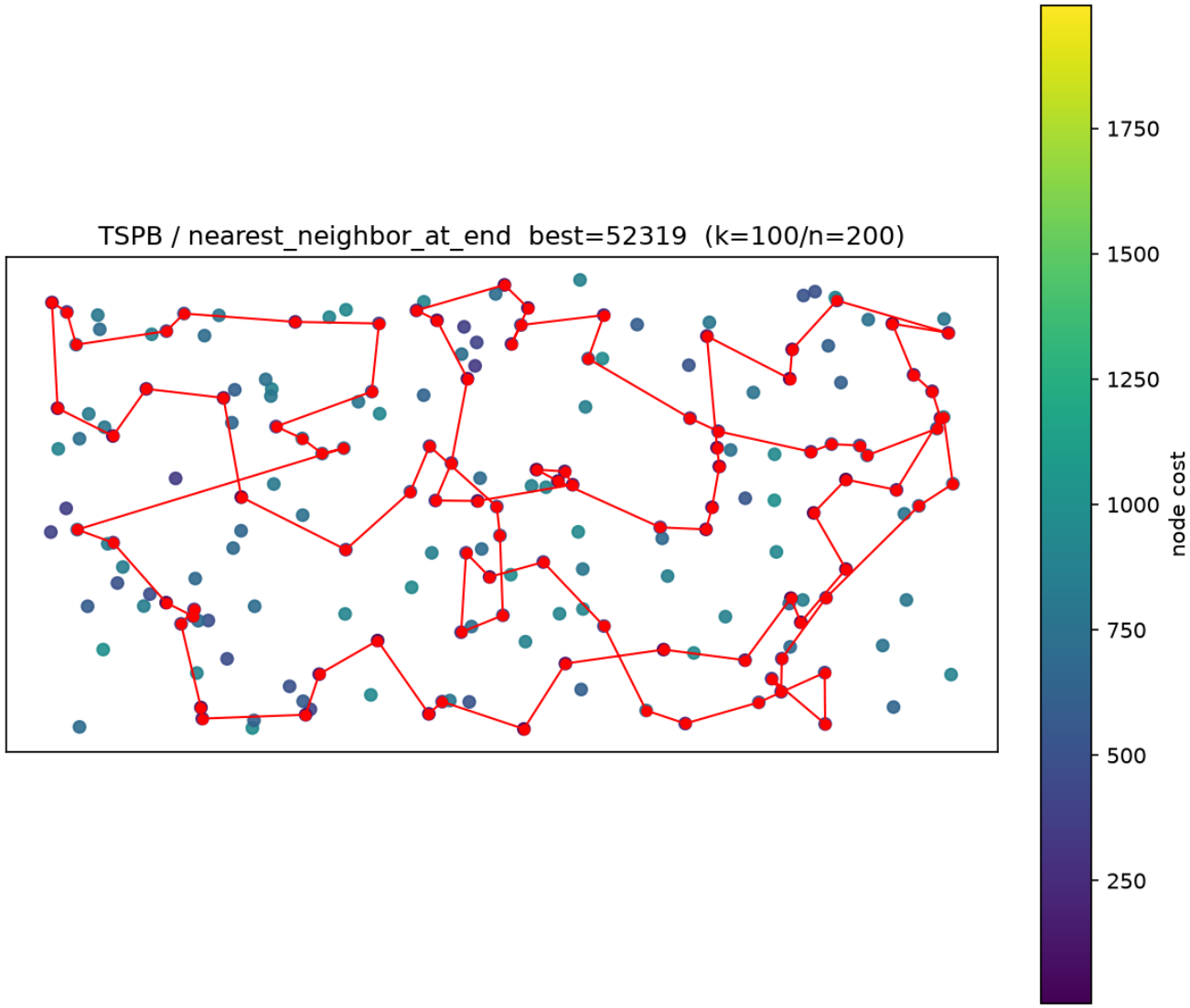
TSPA / greedy_cycle — best=71488



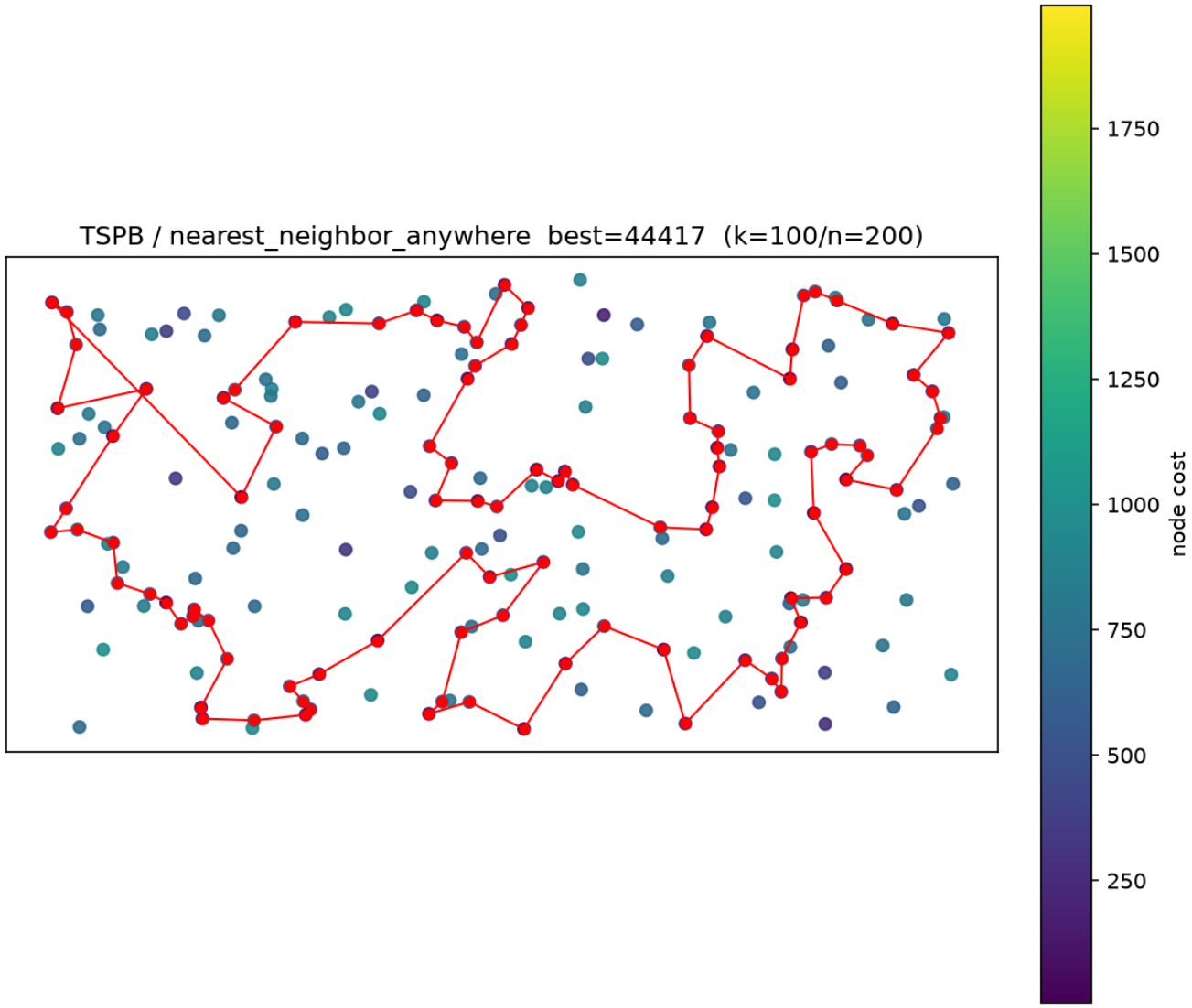
TSPB / random — best=190279



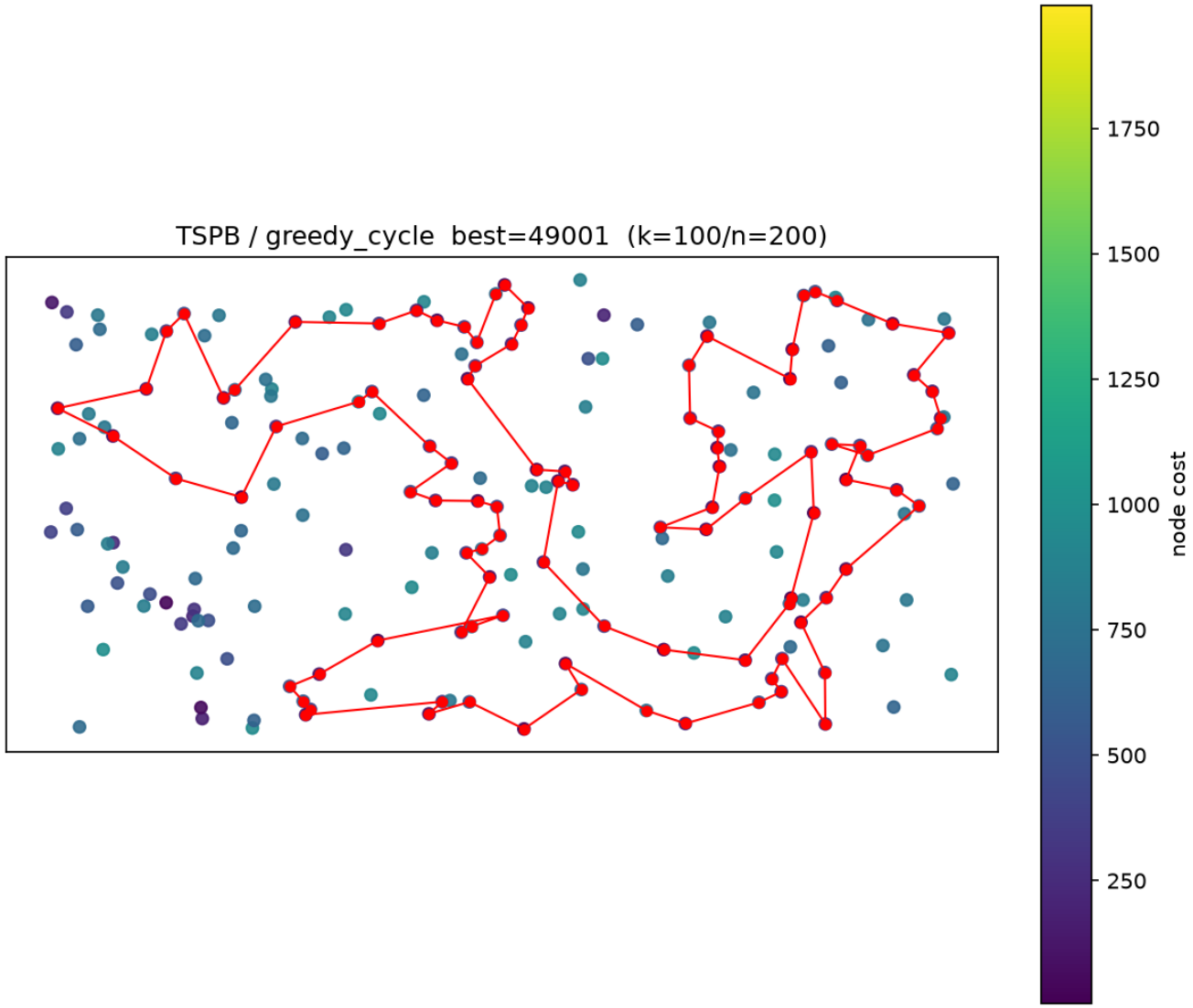
TSPB / nearest_neighbor_at_end — best=52319



TSPB / nearest_neighbor_anywhere — best=44417



TSPB / greedy_cycle — best=49001



4. Best solutions as node index lists (0-based)

TSPA

random:

```
[195, 108, 8, 144, 35, 18, 93, 74, 11, 165, 188, 16, 44, 26, 76, 10, 198, 31, 180, 41, 184, 13, 9, 79,
80, 4, 78, 133, 132, 138, 45, 50, 143, 113, 84, 24, 59, 166, 86, 157, 85, 156, 168, 53, 183, 5, 181,
105, 194, 155, 103, 193, 130, 70, 68, 58, 49, 123, 42, 116, 148, 110, 22, 38, 52, 167, 139, 19, 51,
191, 30, 63, 14, 81, 23, 73, 87, 152, 178, 90, 164, 189, 32, 117, 64, 118, 151, 199, 21, 40, 185, 196,
176, 150, 140, 96, 83, 0, 153, 60]
```

nearest_neighbor_at_end:

```
[124, 94, 63, 53, 180, 154, 135, 123, 65, 116, 59, 115, 139, 193, 41, 42, 160, 34, 22, 18, 108, 69,
159, 181, 184, 177, 54, 30, 48, 43, 151, 176, 80, 79, 133, 162, 51, 137, 183, 143, 0, 117, 46, 68, 93,
140, 36, 163, 199, 146, 195, 103, 5, 96, 118, 149, 131, 112, 4, 84, 35, 10, 190, 127, 70, 101, 97, 1,
152, 120, 78, 145, 185, 40, 165, 90, 81, 113, 175, 171, 16, 31, 44, 92, 57, 106, 49, 144, 62, 14, 178,
52, 55, 129, 2, 75, 86, 26, 100, 121]
```

nearest_neighbor_anywhere:

```
[68, 46, 115, 139, 193, 41, 5, 42, 181, 159, 69, 108, 18, 22, 146, 34, 160, 48, 54, 177, 10, 190, 4,
112, 84, 35, 184, 43, 116, 65, 59, 118, 51, 151, 133, 162, 123, 127, 70, 135, 180, 154, 53, 100, 26,
86, 75, 44, 25, 16, 171, 175, 113, 56, 31, 78, 145, 179, 92, 57, 52, 185, 119, 40, 196, 81, 90, 165,
106, 178, 14, 144, 62, 9, 148, 102, 49, 55, 129, 120, 2, 101, 1, 97, 152, 124, 94, 63, 79, 80, 176,
137, 23, 186, 89, 183, 143, 0, 117, 93]
```

greedy_cycle:

```
[0, 46, 68, 139, 193, 41, 115, 5, 42, 181, 159, 69, 108, 18, 22, 146, 34, 160, 48, 54, 30, 177, 10,
190, 4, 112, 84, 35, 184, 43, 116, 65, 59, 118, 51, 151, 133, 162, 123, 127, 70, 135, 180, 154, 53,
100, 26, 86, 75, 44, 25, 16, 171, 175, 113, 56, 31, 78, 145, 179, 92, 57, 52, 185, 119, 40, 196, 81,
90, 165, 106, 178, 14, 144, 62, 9, 148, 102, 49, 55, 129, 120, 2, 101, 1, 97, 152, 124, 94, 63, 79,
80, 176, 137, 23, 186, 89, 183, 143, 117]
```

TSPB

random:

```
[93, 111, 58, 38, 36, 16, 153, 39, 159, 141, 87, 102, 77, 138, 171, 20, 17, 100, 147, 83, 132, 158,
184, 193, 73, 143, 26, 130, 135, 145, 8, 182, 0, 131, 97, 95, 62, 178, 37, 170, 57, 160, 85, 115, 53,
49, 84, 66, 172, 110, 190, 176, 43, 99, 118, 5, 164, 54, 1, 78, 152, 64, 124, 189, 185, 117, 165, 65,
151, 107, 13, 150, 80, 104, 173, 195, 119, 47, 29, 149, 42, 144, 86, 155, 168, 120, 10, 121, 79, 142,
186, 101, 75, 113, 76, 31, 11, 188, 61, 9]
```

nearest_neighbor_at_end:

```
[16, 1, 117, 31, 54, 193, 190, 80, 175, 5, 177, 36, 61, 141, 77, 153, 163, 176, 113, 166, 86, 185,
179, 94, 47, 148, 20, 60, 28, 140, 183, 152, 18, 62, 124, 106, 143, 0, 29, 109, 35, 33, 138, 11, 168,
169, 188, 70, 3, 145, 15, 155, 189, 34, 55, 95, 130, 99, 22, 66, 154, 57, 172, 194, 103, 127, 89, 137,
114, 165, 187, 146, 81, 111, 8, 104, 21, 82, 144, 160, 139, 182, 25, 121, 90, 122, 135, 63, 40, 107,
100, 133, 10, 147, 6, 134, 51, 98, 118, 74]
```

nearest_neighbor_anywhere:

```
[40, 107, 100, 63, 122, 135, 38, 27, 16, 1, 156, 198, 117, 193, 31, 54, 73, 136, 190, 80, 162, 175,
78, 142, 45, 5, 177, 104, 8, 111, 82, 21, 61, 36, 91, 141, 77, 81, 153, 187, 163, 89, 127, 103, 113,
176, 194, 166, 86, 95, 130, 99, 22, 185, 179, 66, 94, 47, 148, 60, 20, 28, 149, 4, 140, 183, 152, 170,
34, 55, 18, 62, 124, 106, 143, 35, 109, 0, 29, 160, 33, 138, 11, 139, 168, 195, 145, 15, 3, 70, 13,
132, 169, 188, 6, 147, 191, 90, 51, 121]
```

greedy_cycle:

```
[85, 51, 121, 131, 135, 63, 122, 133, 10, 90, 191, 147, 6, 188, 169, 132, 13, 161, 70, 3, 15, 145,
195, 168, 29, 109, 35, 0, 111, 81, 153, 163, 180, 176, 86, 95, 128, 106, 143, 124, 62, 18, 55, 34,
170, 152, 183, 140, 4, 149, 28, 20, 60, 148, 47, 94, 66, 22, 130, 99, 185, 179, 172, 166, 194, 113,
114, 137, 103, 89, 127, 165, 187, 146, 77, 97, 141, 91, 36, 61, 175, 78, 142, 45, 5, 177, 82, 87, 21,
8, 104, 56, 144, 160, 33, 138, 182, 11, 139, 134]
```

5. Solution checking

All 8 best tours return were checked manually with solution-checker spreadsheet and are correct.

6. Source code

https://github.com/skalskidaniel/evolutionary-computation/blob/main/lab1.ipynb

7. Conclusions

1. **Greedy >> random.** Even the weakest greedy (NN-END) improves over the best of 200 random tours by 2.8× (TSPA) and 3.6× (TSPB).
2. **Insertion position matters more than closure awareness on these instances.** NN-ANYWHERE beats NN-END everywhere significantly . Allowing interior insertions removes the forced tail order and avoids paying one huge closing edge. This is the single cheapest algorithmic upgrade (same greedy idea, $0(k)$ positions instead of 1).
3. **Cycle vs. path-anywhere is instance-dependent.** On TSPA, greedy-cycle wins on mean/worst-case, while on TSPB it loses in terms of every measure.
4. **Start-node sensitivity is real but bounded.** Greedy ranges are narrow

(TSPA cycle: 71488–74410, ±2%; TSPB anywhere: 44417–53438, wider, ~20% with one outlier tail). Multi-start (200 runs) is therefore mandatory; single-start greedy would be a lottery.

5. **Cost term is load-bearing.** All deltas include `+C[c]`. Without it, heuristics would chase pure geometry and pick dense regions. Visualizations confirm winners thread through low-cost regions, not just short edges.